IsaacLab Skeleton

```python
def main():
"""Main function."""
env_cfg = RoboticEnvCfg()
env = ManagerBasedEnv(cfg=env_cfg)
# setup base environment

# simulate physics
count = 0
while simulation_app.is_running():
    with torch.inference_mode():
        # reset
        if count % 300 == 0:
            count = 0
            env.reset()
        # sample random actions
        joint_efforts = torch.randn_like(env.action_manager.action)


        # step the environment
        obs, _ = env.step(joint_efforts)

        # update counters
        count += 1
```

High-Level Class managing simulation environments, Observations, Actions, ..

Resets Environment Every 300 steps

Generate the action for next step. Here's a simple example, so we generate random torques to be applied to the robot. In a RL context, as an example, we would put here the action computed by the agent

Computes next simulation step (update the scene to next timestamp dt)

ManagerBasedEnv

- High-level interface that manages the simulated environment. It contains:
  - **Scene:** The physical Simulation of the scene
  - **Observation Manager:** manages sensor reading generate from the scene
  - **Action Manager:** manages processes the raw actions sent to the environment and converts them to low-level commands that are sent to the simulation (e.g. commanded joint torques, joint positions, …)
  - **Event Manager**: orchestrates operations triggered based on simulation events (e.g. resetting the scene to a default state, applying random pushes to the robot at different intervals of time, ..)

ManagerBasedEnv

- High-level interface that manages the simulated environment. It contains:
  - **Scene:** The physical Simulation of the scene
  - **Observation Manager:** manages sensor reading generate from the scene
  - **Action Manager:** manages processes the raw actions sent to the environment and converts them to low-level commands that are sent to the simulation (e.g. commanded joint torques, joint positions, …)
  - **Event Manager**: orchestrates operations triggered based on simulation events (e.g. resetting the scene to a default state, applying random pushes to the robot at different intervals of time, ..)

ManagerBasedEnvConfig

- The configuration of the environment. It contains all the required information to run the scene simulation, observation and action managers
- An Example:

```python
@configclass
class RoboticEnvCfg(ManagerBasedEnvCfg):
    # scene settings
    scene: RoboticSoftCfg = RoboticSoftCfg(num_envs=args_cli.num_envs, env_spacing=2.5)
    # Basic settings
    observations = ObservationsCfg()
    actions = ActionsCfg()
    events = EventCfg()

    def __post_init__(self):
        """Post initialization."""
        # viewer settings
        self.viewer.eye = [4.5, 0.0, 6.0]
        self.viewer.lookat = [0.0, 0.0, 2.0]
        # step settings
        self.decimation = 4  # env step every 4 sim steps: 200Hz / 4 = 50Hz
        # simulation settings
        self.sim.dt = 0.005  # sim step every 5ms: 200Hz
```

Scene

```
scene: RoboticSoftCfg = RoboticSoftCfg(num_envs=args_cli.num_envs, env_spacing=2.5)
```

- The scene Config contains the information needed to run the physical simulation. This should contain all the info regarding the objects in the scene, their physical properties, their relative positions, as well as scene lighting, ..
- As an example, the scene below represent a scene with a ground floor, a table, a robot, and lighting simulating sunlight (DomeLight):

```
@configclass
class RoboticSoftCfg(InteractiveSceneCfg):
    # ground plane
    ground = AssetBaseCfg(
        prim_path="/World/defaultGroundPlane",
        init_state=AssetBaseCfg.InitialStateCfg(pos=[0, 0, -1.05]),
        spawn=sim_utils.GroundPlaneCfg())

    # lights
    dome_light = AssetBaseCfg(
        prim_path="/World/Light", spawn=sim_utils.DomeLightCfg(intensity=3000.0, color=(0.75, 0.75, 0.75))
    )
    table = AssetBaseCfg(
        prim_path="{ENV_REGEX_NS}/Table",
        init_state=AssetBaseCfg.InitialStateCfg(pos=[0.5, 0, 0], rot=[0.707, 0, 0, 0.707]),
        spawn=sim_utils.UsdFileCfg(usd_path=f"{ISAAC_NUCLEUS_DIR}/Props/Mounts/SeattleLabTable/table_instanceable.usd"),
    )
    # articulation
    # -- Robot
    robot: ArticulationCfg = FRANKA_PANDA_CFG.replace(prim_path="{ENV_REGEX_NS}/Robot")

   …
```

Observations

```
observations = ObservationsCfg()
```

- The observation config contains the information to retrieve observation from the scene.
- The Observation config can either be a class or a dict of *ObservationGroupCfg*. Each ObservationGroupCfg contains a set of ObservationTermCf. An ObservationTermCfg contains the function to be called to read the information. The function should take take the environment object and any other parameters as input and return the observation as torch float tensors of shape (num_envs, obs_term_dim). In the example below, we use the pre-defined mdp.<function>. These functions read and return the desired variables from the scene. Note: If not specified otherwise, the mdp functions in the example read the desired variable for the scene asset with name “robot”, by default. If the device you want to read your variable from is different, make sure to specify this (e.g. by adding to each term `joint_pos.params["asset_config"] = SceneEntityCfg(<desired_asset_name>)`)

```python
class ObservationsCfg:
    """Observation specifications for the MDP."""

    @configclass
    class PolicyCfg(ObservationGroupCfg):
        """Observations for policy group."""

        joint_pos = ObservationTermCfg(func=mdp.joint_pos_rel)
        joint_vel = ObservationTermCfg(func=mdp.joint_vel_rel)
        object_position = ObservationTermCfg(func=mdp.object_position_in_robot_root_frame)
        target_object_position = ObservationTermCfg(func=mdp.generated_commands, params={"command_name": "object_pose"})
        actions = ObservationTermCfg(func=mdp.last_action)

        def __post_init__(self):
            self.enable_corruption = True
            self.concatenate_terms = True

    # observation groups
    policy: PolicyCfg = PolicyCfg()
```

ManagerBasedRLEnv

```
class ManagerBasedRLEnv(ManagerBasedEnv, gym.Env)
```

- Superclass for the manager-based workflow reinforcement learning-based environments.
- This environment is a wrapper for handling multiple subenvironments for parallel training. Each observation from the environment is a batch of observations for each sub- environments. The method *step()* is also expected to receive a batch of actions for each sub-environment.

- Custom Environments: https://isaac-sim.github.io/IsaacLab/main/source/tutorials/03_envs/run_rl_training.html